温州理工学院

《数学建模与实验》课程大论文

（参加 2022 年电工杯数学建模竞赛团队参赛作品）

选做题号：B（填写 A 或 B）

	参赛队员 1	参赛队员 2	参赛队员 3
姓名	於豪祺	王帅骏	应维
学号	20219130217	20219111103	20219111318
学院	数据科学与人工智能学院	数据科学与人工智能学院	数据科学与人工智能学院
班级	20 应用统计二班	20 计算机一班	20 计算机三班
电话	19858817237	18757816912	19941231600
Email	1768110051@qq.com	1243462510@qq.com	2062710664@qq.com

温州理工学院数据科学与人工智能学院

《数学建模与实验》课程组制

2022 年 5 月 29 日

报名序号： 1735

赛题题目：B

优化物资配送的数学模型

摘要

随着科技水平的提升以及 5G 网络普及，无人机的应有越来越广泛，一种新的配送方式：“配送车辆+无人机”产生了。在发生特大事故，道路损、阻断、封闭，导致车辆无法将应急物资配送到指定地点时，“配送车辆+无人机”的配送模式就可以发挥很大作用。这个模式实在配送车辆对某受灾地点进行配送时，无人机也对周围地点进行配送，并在配送完成后返回车辆，无人机可在车辆无法通行的地方进行配送这就大大提高了配送效率。

针对问题一，在只有车辆进行配送的情况下，对给定的 14 个地点进行配送。我们引入 **0-1 变量**，用来表示两地之间是否用车辆进行配送，再由这些变量与两地之间的矩阵矩阵构成最短路径模型。通过 **Floyd 算法**求出两个地点的最短路径，然后根据**局部最优**算法完成对模型的求解。求得的地点顺序为 $9 \to 6 \to 4 \to 10 \to 14 \to 13 \to 8 \to 12 \to 11 \to 1 \to 5 \to 2 \to 7 \to 3 \to 9$，总距离为 645 公里，时间为 12.9 小时。

针对问题二，在“配送车辆+无人机”的配送模式下，对 14 个地点进行配送。延用问题 1 的最短路径模型，需要再次添加一个 **0-1 变量**用来表示两地之间是否用无人机配送，还要添加无人机的约束条件，由这些构成最短路径模型。将平均配送时间定义为适应度，通过**遗传算法**完成对模型的求解。求得车辆配送最优方案总里程为 453 公里，最短配送时间使 9.06 小时。

针对问题三，在问题 2 的基础上，将车辆的最大承载从 1000 变为了 500。继续使用问题 2 的最短路径模型，需要添加车辆最大承载的约束条件，完成最短路径模型构建。用**聚类算法**先将原本的 14 个点分为两类，代表车辆配送的两个批次。通过遗传算法完成对模型的求解。求得将地点 9，6，4，10，14，12，13 分为一个配送批次，将地点 9，8，7，5，2，3，1，11 分为另一配送批次。并求得总的车辆最优方案总里程 451 公里，总的最短配送时间是 9.02。

针对问题四，在“配送车辆+无人机”的配送模式下，对 30 个地点进行配送，要确定两个物资集中点。先通过聚类算法将 30 个点分为两堆，使得这两堆的总距离最短。再根据问题 3 的求解过程，取得出这两堆下总的配送最短时间。通过迭代，将时间短的保留，使总时间趋于稳定，最后得出结果。求得两个最佳的物资集中点使 21 和 22，总配送时间是 5.4 小时。

关键字： **0-1 变量** **Floyd 算法** **局部最优** **遗传算法** **聚类算法**

一 问题重述

1.1 引言

随着科技水平的提升以及 5G 网络普及，无人机的应有越来越广泛，一种新的配送方式：“配送车辆+无人机”产生了。在发生特大事故，道路损、阻断、封闭，导致车辆无法将应急物资配送到指定地点时，“配送车辆+无人机”的配送模式就可以发挥很大作用。这个模式实在配送车辆对某受灾地点进行配送时，无人机也对周围地点进行配送，并在配送完成后返回车辆，无人机可在车辆无法通行的地方进行配送这就大大提高了配送效率。

1.2 问题的提出

问题 1：

题目给出了 14 个受灾地点，其中物资都集中在 9 号地点，给了各地点之间的路线情况与各地点之间的距离，以及各地点需要的配送的物资重量，这些信息。在只有车辆进行配送，无人机不参与，且只有一辆配送车辆，配送车辆的最大承重为 1000 千克的情况下，建立出数学模型，给出最佳的配送方案。

问题 2：

采用“配送车辆+无人机”配送模式，在问题 1 的基础上，对地点之间的路线进行了更改。问题 1 中车辆可以进行配送的路线，无人机也可以配送，并且无人机还可以工作于一些车辆无法进行配送的路线。但是一辆车只能携带一架无人机，且无人机的最大承重为 50 千克，飞行速度为 75 公里/小时，单次最长飞行 70 分钟。对此建立出数学模型，给出最佳的配送方案

问题 3：

在问题 2 的基础上，将配送车辆的最大载重量调整为 500 千克，其他条件发生变化。对此建立出数学模型，给出最佳的配送方案。

问题 4：

题目重新给出了 30 个受灾地点，并给出了各地点之间的路线情况与各地点之间的距离，以及各地点需要的配送的物资重量。并采用“配送车辆+无人机”配送模式，配送车辆的最大承重还是 500 千克。在给出的 30 个受灾地点中，计划设置两个物资集中点，并有两辆配送车辆以及两架无人机对应这两个物资集重点。需要建立出数学模型，确定这两个物资及重点的最佳位置，使得配送方案达到最佳。

二 模型假设

（1）忽略无人机自身重量影响，忽略载重对车辆影响。
（2）配送车辆和无人机行驶速度保持不变。
（3）不考虑配送车无人机故障等意外情况。
（4）无人机不会连续向同一个地方配送物资。

三 符号说明

符号	说明
M	地点数
O_i	第 i 个地点与其他可以互通地点的数量
d_{ab}	从 a 点到 b 点的距离
x_{ab}	表示车辆是否进行配送
D_{ij}	各地点之间的关系
$\mathbf{x}_{ij}$	表示两地之间是否用车辆进行配送
y_{ab}	表示无人机是否进行配送
n	无人机飞行的次数
q_b	b 点所需物资数量
m	车辆配送的次数
Y_{ij}	表示两地之间是否用无人机进行配送

四 问题分析

4.1 问题一的分析

在只有 1 辆车进行配送的情况下，对 14 个地点进行物资配送。其中物资都集中在 9 号地点，意味着需要从 9 号地点出发对所有地点进行物资配送。需要我们建立一个数学模型使配送路线达到最短，可以根据目标函数、决策变量、约束条件，建立出一个最短路径模型。由于要找出最短的路线，我们可以使用局部最优算法，找到这个点到下一个点的最短距离，从而完成问题 1 的求解

4.2 问题二的分析

在有一辆车和一架无人机配送的情况下，对 14 个地点进行配送。整个配送物资的流程是配送车辆从第 9 个地点出发，与无人机一起完成物资配送。此时可以先根据点与点之间距离、无人机续航和平均速度初步计算出哪些地点可以使用无人机配送，将其纳入可行解构范围，对无人机可到达地方进行标识（与汽车到过的位置标识一样），但是优化原理还是一样的，只不过因为无人机的加入，能使地点能提前到达而已。

4.3 问题三的分析

第二问的基础上增加了限制条件，车辆最大载重从 1000kg 变成 500kg，此时车辆必须回到一次物资聚集地，那么我们需要对地方进行分成两组，使用算法将所有地点分为两类，使得两类组内距离总和较小。目标函数、决策变量、约束条件，与第二题类似，那么我开门可以套用第二题的遗传算法模型，建立完成一次整体配送的数学模型，并给出最优方案。

4.4 问题四的分析

三问的基础上增加限制条件，新增地点、车辆无人机和物资需求量变化。但配送模式并不改变，那么我么们可以继续套用第二问的算法模型先计算两两地点之间的最短距离、最短路径和最短时间计算在无人机续航里程内的所有可飞行路径。采取“配送车辆+无人机”的配送模式。结合附件 3，再用第三问的算法，对两个物资配送范围进行计算，分别运用问题的方法对两个聚类进行计算。即可得出每个分类每一趟的最优方案。

五 模型假设和求解

5.1 问题一的模型建立和求解

5.1.1 最短路径模型建立

由题意可知，我们需要建立一个数学模型，使得车辆在 14 个地点中的配送路线可以达到最短。同时给出了约束条件：只有车辆进行配送，车辆的最大承载为 1000 千克。经过计算发现，所有地区所需的物资总量为 762 千克，不超过 1000 千克，则不需要考虑承重这一约束条件。

为建立出最短路径模型，我们引入 0-1 规划，来构建各点之间的可达矩阵 X_{ij}，用来表示两地之间是否用车辆进行配送。可达矩阵为：

$$X_{ij}\begin{cases}0，\text{i 到 j 不可达}\\ 1，\text{i 到 j 可达}\end{cases} \tag{1}$$

并用一个变量表示两地之间的距离,构成距离矩阵 D_{ij},如果两地之间不可达，两地间的距离则用一个无穷大值来表示，此处用 INF 表示。

$$D_{ij}=\begin{cases}INF & \text{i到j不可达}\\ d & \text{i到j可达}\end{cases} \tag{2}$$

二者相乘并累加的最小值即为最短路径，具体可见如下模型。

目标函数：在只有配送车辆进行配送的情况下，完成一次整体的应急物资配送所经历的路程最短。

决策变量：配送目标地点，各地点之间的路线情况，各地点之间的距离。

建出模型：

$$\min D=\sum_{i=1}^{M}(\sum_{a=0}^{O_i}\sum_{b=0}^{O_i}d_{ab}x_{ab}) \tag{3}$$

其中O_i表示，第 i 个地点与其他可以互通地点的数量，d_{ab}为从 a 点到 b 点的距离，x_{ab}是一个 0-1 变量，若车辆从第 a 个地点到第 b 个地点为 1，否则为 0。

5.1.2 Floyd 算法求两点最短路径

Floyd 算法是一种利用动态规划的思想寻找给定的加权图中多源点之间最短路径的

算法，其目标是寻找从节点 i 到节点 j 的最短路径。[1]

首先我们需要构建一个距离矩阵 D_{ij} 来表示各地点之间的关系,即以上（2）式的表达式。

我们通过一个点为基础，不断去确定更新两地间的最短路径。循环比较 D_{vw} 和 $D_{vk} + D_{kw}$ 最小值，如果 $D_{vk} + D_{kw}$ 为更小值，则把 $D_{vk} + D_{kw}$ 覆盖保存在 D_{vw} 中。[2]

流程示意图如下：

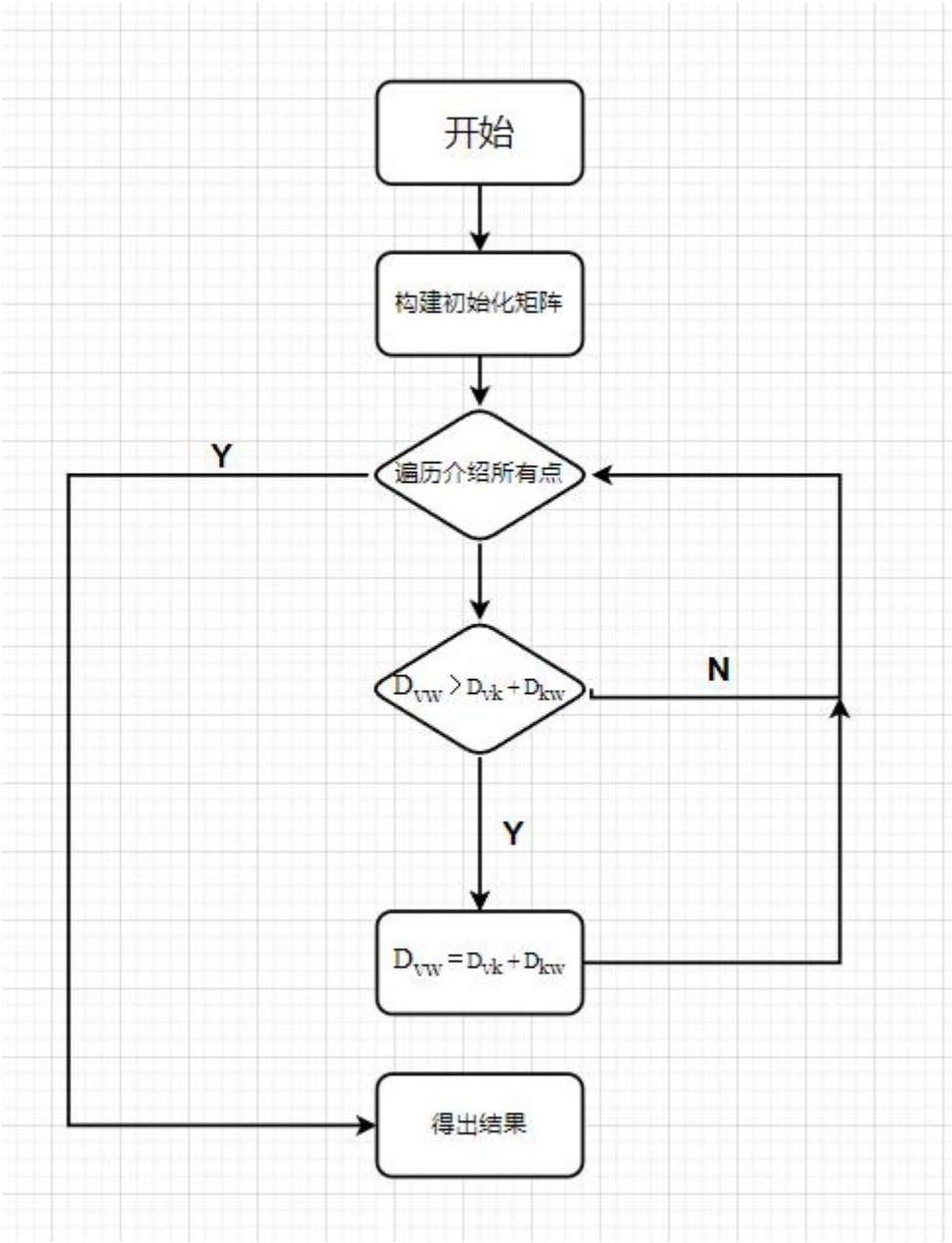

图 1 Floyd 算法流程

5.1.3 基于局部最优算法对模型的求解

在上部分中，已经建立出了最短路径模型，接下来就是对最短路径模型的求解。先根据附件 1 的内容，建立出一个距离矩阵，建立出一个全新的 0-1 矩阵，用来表示两地

之间是否可以用车辆进行配送，如果可以用车辆进行配送则为 1，不可以用车配送则为 0。

使用局部最优算法对模型进行求解，局部最优算法是指，在对问题求解时，总是做出在当前看来时最好的选择。也就是说，不从整体最优上加以考虑，算法得到的是某种意义上的局部最优解，拥有贪心的思想。

一个问题的最优解包含其子问题的最优解。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。

在本题中的局部最优即为该点距离下一点最近的点。通过程序找出最近的点，并对该地点进行标记。然后在原来的路线情况上将找出的点进行处理，将该点在原来路线上清除，表示该点的配送任务已经完成。

由于该点的清除，本来因为中间隔着该点而不能直接相通的两点，现在可以直接相通。重新计算两点可以相通之后的距离，更新原来的距离矩阵。同时更新 0-1 矩阵，将有关该点的路线都变为 0，现在可以直接相通的两点的路线变为 1。

在该点两端的距离则可以表示为 D_{ab}。

$$D_{ab} = \min(D_{ia} + D_{\mathrm{ib}}, D_{ab}) \tag{4}$$

该算法通过不断去找下一路径最短的地点，并将当前点从地图中抹去，联通相连的其他点，一直到最后只剩下一个点。

通过求解，我们得到了最有的地点顺序为：

$$9 \to 6 \to 4 \to 10 \to 14 \to 13 \to 8 \to 12 \to 11 \to 1 \to 5 \to 2 \to 7 \to 3 \to 9$$

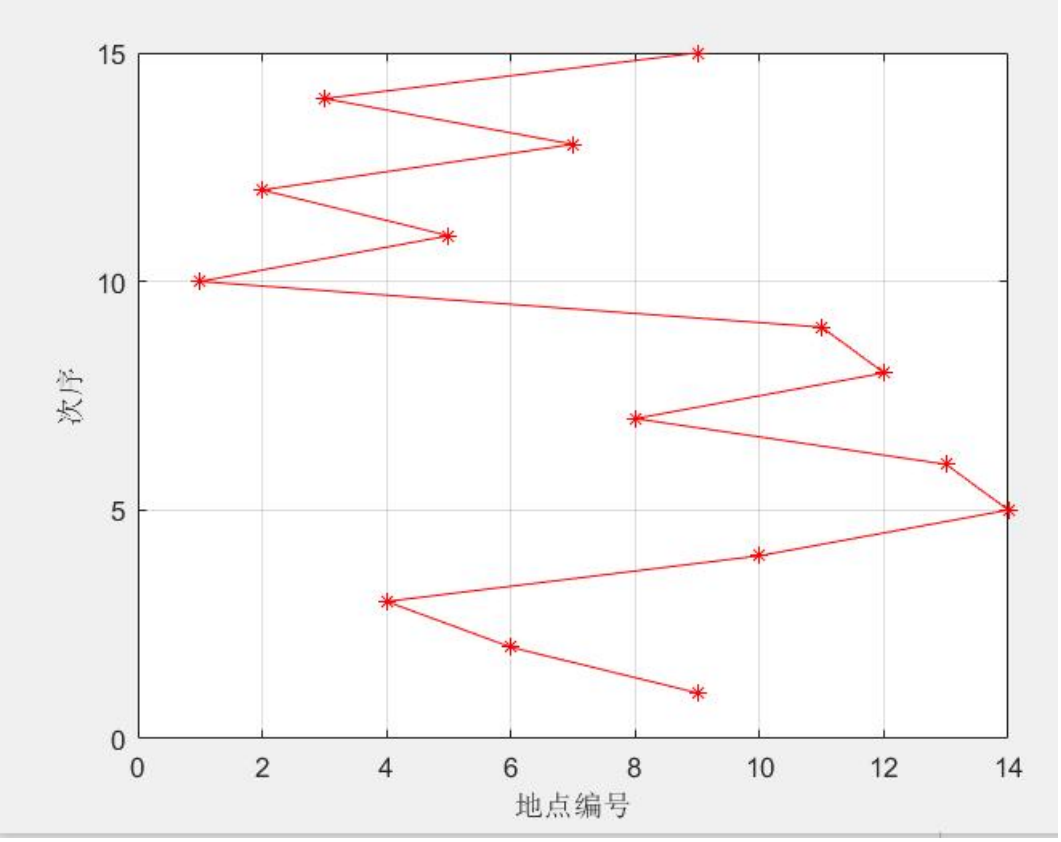

图 2 地点顺序图 1

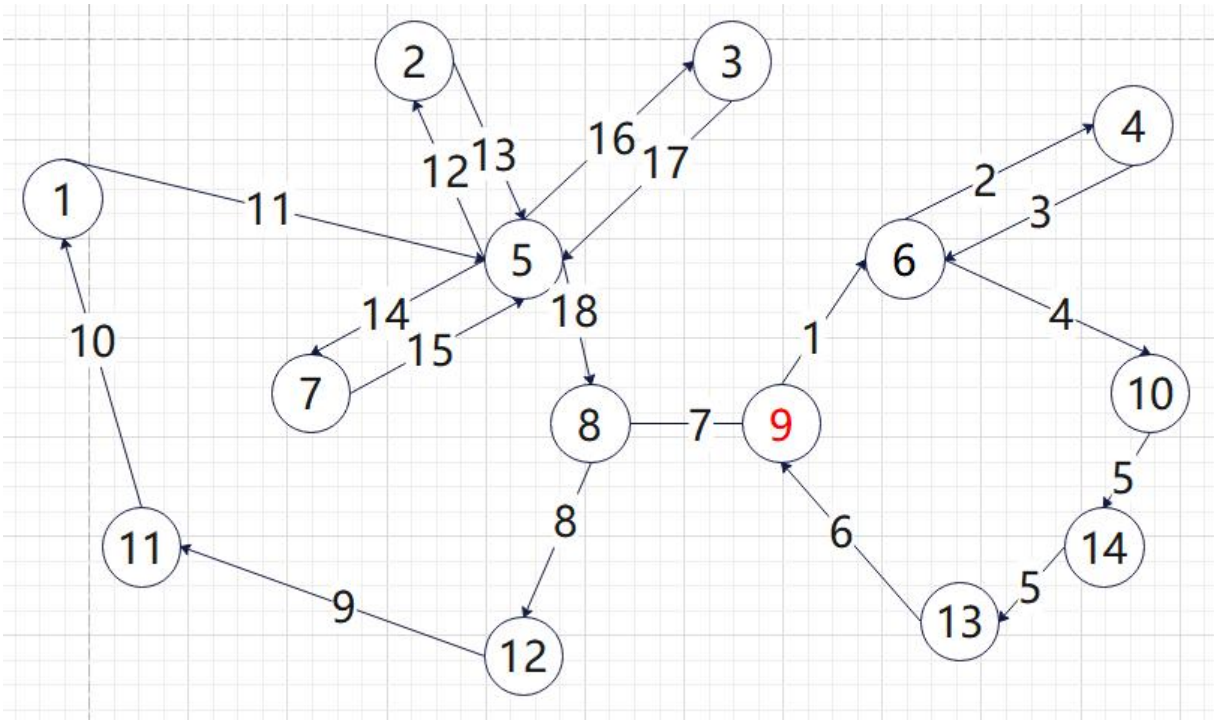

图 3 地点顺序图 2

求得总距离 645，时间为 12.9 小时。

5.2 问题二的模型建立和求解

5.2.1 “无人机-车辆”模式的最短路径模型建立

在问题 1 只有车辆进行派送的情况下，加入无人机与车辆协同派送。在问题 1 原有的约束条件，添加上无人机的约束条件，无人机的最大承载为 50 千克，且无人机单次只能飞行 87.5 公里。在这里我们可以再次引入一个 0-1 变量，用来表示两地之间是否使用无人机进行配送。

引入 0-1 规划，来构建各点之间的可达矩阵 Y_{ij}，用来表示两地之间是否用无人机进行配送。可达矩阵为：

$$Y_{ij}\begin{cases}0，\text{i 到 j 不可达}\\ 1，\text{i 到 j 可达}\end{cases} \tag{5}$$

具体如下。

目标函数：在“配送车辆+无人机”的配送模式下完成一次整体的应急物资配送所经历的路程最短。

决策变量：配送目标地点，各地点之间车辆可通行的路线情况，各地之间无人机可通行的路线情况，各地点之间的距离。

约束条件：无人机一次最大承载为 50 千克，无人机单次只能飞行 87.5 公里

建出模型：

$$\min D=\sum_{i=1}^{M}(\sum_{a=0}^{O_i}\sum_{b=0}^{O_i}d_{ab}x_{ab}+\sum_{a=0}^{O_i}\sum_{b=0}^{O_i}d_{ab}y_{ab}) \tag{6}$$

满足条件：

$$\sum_{1}^{n}q_b y_{ab}\le 50 \tag{7}$$

$$\sum_{1}^{n}d_{ab}y_{ab}\le 87.5 \tag{8}$$

其中O_i表示，第 i 个地点与其他可以互通地点的数量，d_{ab}为从 a 点到 b 点的距离，x_{ab}是一个 0-1 变量，若车辆从第 a 个地点到第 b 个地点为 1，否则为 0。其中y_{ab}是一个 0-1 变量，若无人机从第 a 个地点到第 b 个地点为 1，否则为 0。n 表示无人机飞行的次数，q_b表示 b 点所需的物资重量。

5.2.2 基于遗传算法的模型求解

遗传算法试图找到给定问题的最佳解。达尔文进化论保留了种群的个体性状，而遗传算法则保留了针对给定问题的候选解集合。这些候选解结果迭代评估，用于创建下一代解。更优的解有更大的机会被选择，并将其特征传递给下一代候选解集合可以更好地解决当前的问题。

记录配送车辆和无人机的可行路径，在记录无人机的可行路径时，要考虑到求解的实际需要，要计算无人机续航里程内所有可飞行路径。这之中要先将大于无人机最大航程的路径设为无穷大，只留下无人机可以往返的路径，从第一个路径进行判断，如果新路径的距离是小于无人机续航的，那么就可以记录路径。

找出哪些地点可以用无人机进行配送。如果无人机和车辆一起配送同一个地方，由于无人机的行驶速度比车辆行驶速度快，这样会导致等待的时间增加。所以假设无人机不会连续向同一个地点配送，即该点的物资需求量不能超过无人机的最大承载 50 千克。建立两个新的矩阵，用来存放无人机能配送的地点与无人机不能配送的地点，结果见下表。

表 1

无人机可配送									
地点	1	3	4	6	8	11	12	13	14

表 2

无人机不可配送

地点	2	5	7	9	10

将数据进行初始化，以长度为 8 的二进制数字表示无人机可以配送的 8 个地点，0 表示不配送该地点，1 表示配送。产生行为群体大小，列为无人机可配送的地点个数的零矩阵，并对矩阵的每个单元进行圆整，产生出新的初始种群。

将适应度定义为平均配送时间，计算出群体中每个个体的适应度，然后求出群体中适应值最大的个体以及其对应的适应值

最后计算出最佳个体的最优配送方案，最优配送方案的总里程以及最短的配送时间。建立四个全新矩阵来存放，最短配送时间、车辆最优配送方案总里程、车辆最有配送方案、无人机最优配送方案这些数据。使用遗传算法计算出车辆的最佳配送方案，先还原路径原先的序号，并对路径进行补充，在这个路径中不会出现重复的点，但是实际上有的两个点之间没有直接连通，需要先到其他点。在车辆的最佳配送路线基础上计算无人机是否可以完成指定地点的配送任务如果无人机配送的地点在最佳配送路线中出现，则此地点无需无人机配送，修改种群中的个体，如果一个可能的无人机路径包含无人机配送的地点，则此路径可能正好在车辆的最佳路径上，在车辆最优配送路径中寻找出符合要求的包含本次无人机配送的地点的无人机路径。最后计算出各时间，相加得到需要的总时间。

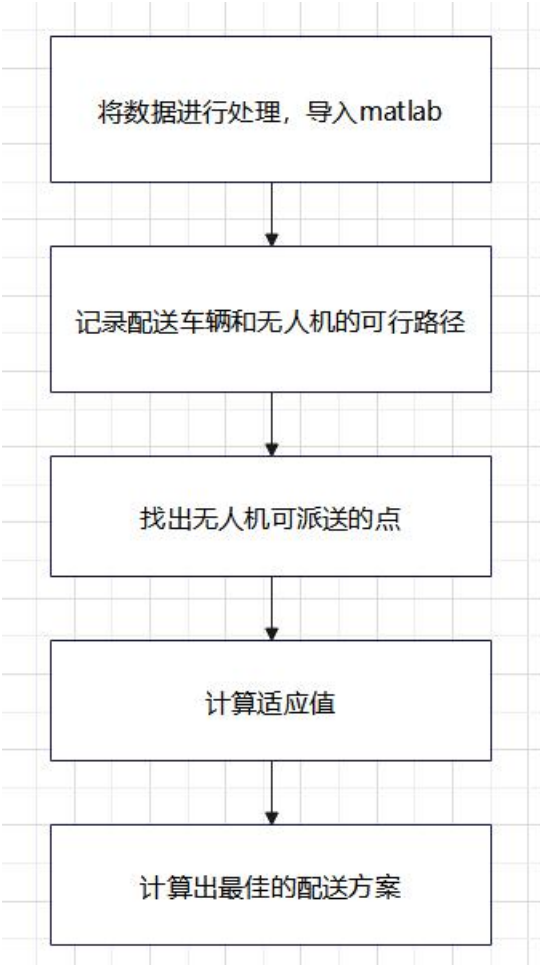

图 4 问题 2 求解流程

通过 matlab 编码运行，得出结果如下。
车辆配送最优方方案总里程为 453 公里，最短的配送时间是 9.06 小时。

车辆行驶路径：

$$9 \to 14 \to 10 \to 6 \to 4 \to 6 \to 5 \to 2 \to 5 \to 1 \to 7 \to 8 \to 12 \to 9$$

无人机行驶路径：

$$4 \to 3 \to 2 \quad 1 \to 11 \to 1 \quad 7 \to 8 \to 13 \to 8$$

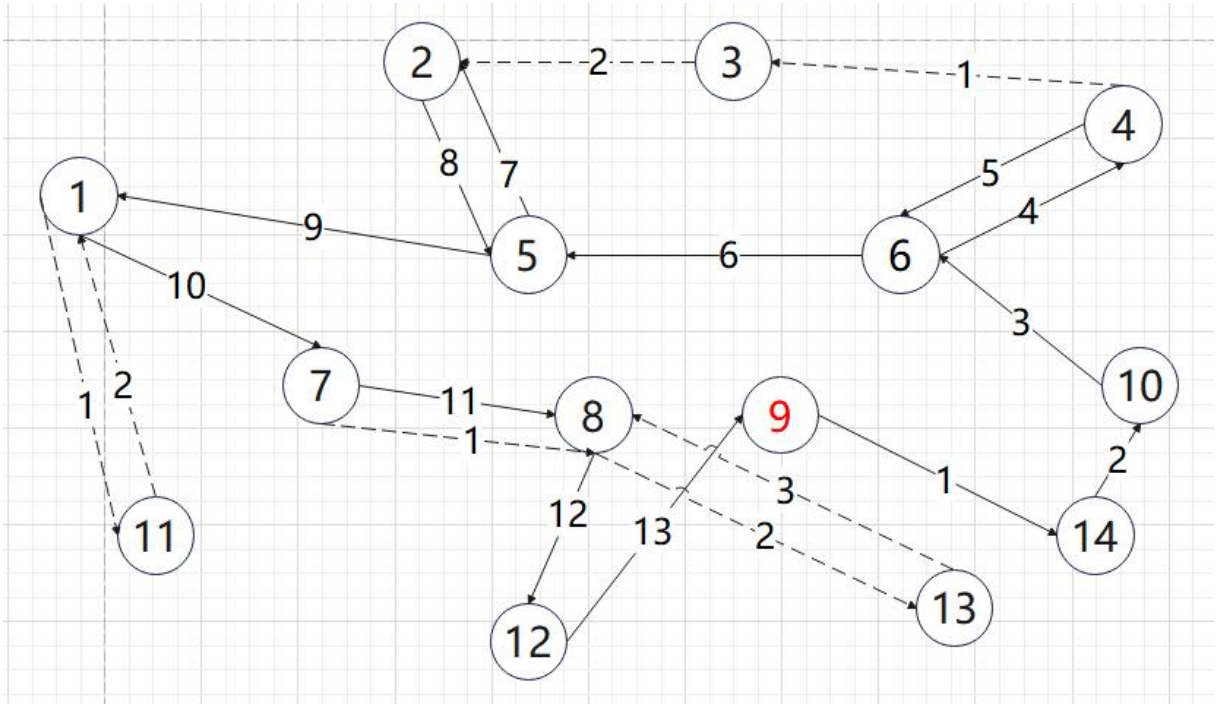

图 5 地点顺序图

5.3 问题三的模型建立和求解

5.3.1 增加约束条件的最短路径模型建立

在问题 2“配送车辆+无人机”的配送模型下，改变一个约束条件，车辆的最大承重从 1000 千克变为 500 千克

目标函数：在“配送车辆+无人机”的配送模式下完成一次整体的应急物资配送所经历的路程最短。

决策变量：配送目标地点，各地点之间车辆可通行的路线情况，各地之间无人机可通行的路线情况，各地点之间的距离。

约束条件：无人机一次最大承载为 50 千克，无人机单次只能飞行 87.5 公里，车辆的最大承载为 500 千克。

建出模型：

$$\min D=\sum_{i=1}^{M}(\sum_{a=0}^{O_i}\sum_{b=0}^{O_i}d_{ab}x_{ab}+\sum_{a=0}^{O_i}\sum_{b=0}^{O_i}d_{ab}y_{ab}) \tag{6}$$

满足条件：

$$\sum_{1}^{n}q_b y_{ab}\le 50 \tag{7}$$

$$\sum_{1}^{n}d_{ab}y_{ab}\le 87.5 \tag{8}$$

$$\sum_{1}^{m}q_b x_{ab}\le 500 \tag{9}$$

其中O_i表示，第 i 个地点与其他可以互通地点的数量，d_{ab}为从 a 点到 b 点的距离，x_{ab}是一个 0-1 变量，若车辆从第 a 个地点到第 b 个地点为 1，否则为 0。其中y_{ab}是一个 0-1 变量，若无人机从第 a 个地点到第 b 个地点为 1，否则为 0。n 表示无人机飞行的次数，q_b表示 b 点所需的物资重量。m 表示车辆配送的次数。

5.3.2 聚类算法分类

在问题 2 的求解过程下，加入全新的约束条件，将车辆的最大承载由 1000 千克变为 500 千克。由于所有地点物资所需的总量为 762 千克，所以车辆总配送次数从一次变为两次，即需要将所有地点分为两类，在这里我们使用聚类算法。

聚类算法作为一种无监督学习方法， 通过将抽象集合分成不同簇类对象，实现对数据 稀疏和稠密区的快速划定与识别，发现独立簇、独 立点，挖掘数据背后的数理关系。[4]

简而言之，聚类的目标是将具有相似特征的群体隔离开来，并将其分配到集群中。在本题中，我们通过构建聚类算法，来使得每类组内距离总和为最小的。

基于类心的聚类被认为是最简单的聚类算法之一，但却是创建聚类和为其分配数据点的最有效方法。基于质心的聚类背后的直觉是，聚类的特征在于中心向量，并且将与这些向量非常接近的数据点分配给各个 聚类。这些聚类方法组使用各种距离度量来迭代地测量 聚类和特征类心之间的距离。它们是欧几里得距离，曼哈顿距离或明可夫斯基距离。事实证明，在处理大型数据集时，它比分层聚类更有价值。同样，由于其 实现和解释的简单性，这些算法具有广泛的应用领域，即市场细分、客户细分、文本主题检索、图像细分等。

首先通过随机选择 k 个点作为类心，并且设置迭代次数，通过一次次的迭代，将类心更新，得到一个最适合的类，直至迭代次数结束。

第二步，在找到类心后，在新类中继续寻找类心，重复第一步的步骤。

第三步，如果新的类心与原来的没有发生改变，那么说明总距离没有发生改变，则当前的解为最适解，聚类结束，反之，继续循环。

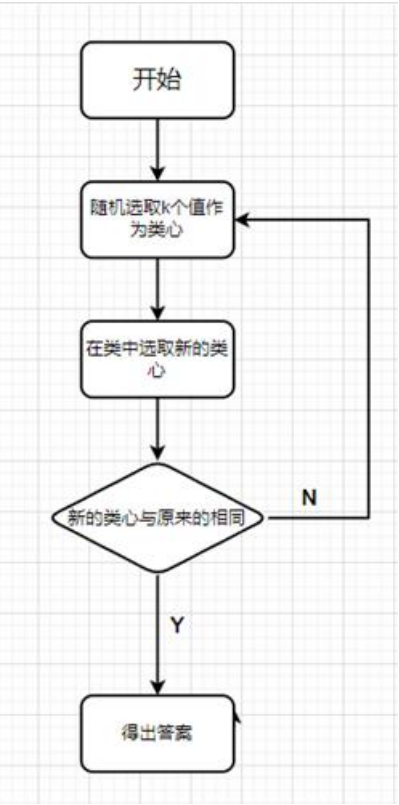

图 6 **聚类算法流程图**

为使两类组内距离达到最小，使用聚类算法将所有地点分为两类。先随机找一个解，作为初始的可行解，由于初始解会影响聚类的速度和聚类结果，会使结果变成局部最优解。所以我们选择通过多次随机选取初始解找到较优的结果。将样本点归在离的最近的类心，在类内找寻新的类心，新类心的条件是离这些点的总距离的最短，即为数据点的中心。由于类心的改变会导致总距离的同步变化，所以通过总距离的变化来判断类心是否有变化，以此最后确定出分开的类点。

表 3

第一类							
地点	9	6	4	10	14	12	13

表 4

第二类

地点	9	8	7	5	2	3	1	11

5.3.3 建模求解

在问题 2 求解过程的基础上进行修改，在计算最佳方案之前，我们需要先把所有地点分为两类。在上部分中，我们已经将所有地点用聚类算法分为了两类。然后分别使用遗传算法计算出两个类次的最优配送方案。

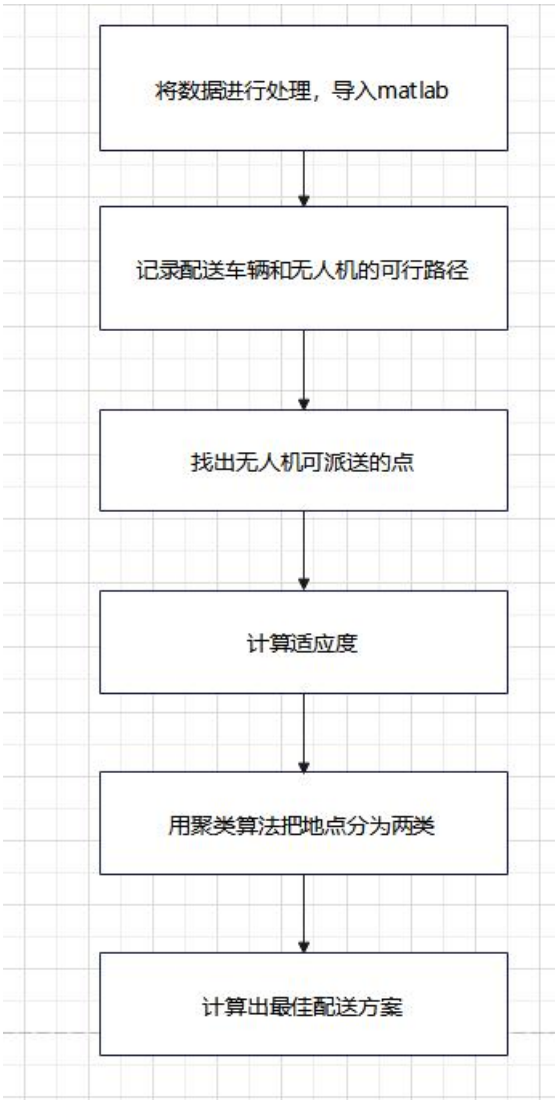

图 7 问题 3 求解流程

通过 matlab 编码运行，得出结果如下。

第一趟车辆配送最优方案总里程为 186 公里，最短的配送时间是 3.72 小时。

车辆行驶路径：

$$9 \to 6 \to 4 \to 6 \to 10 \to 14 \to 9$$

无人机行驶路径：

$$9 \to 12 \to 9 \quad 9 \to 13 \to 9$$

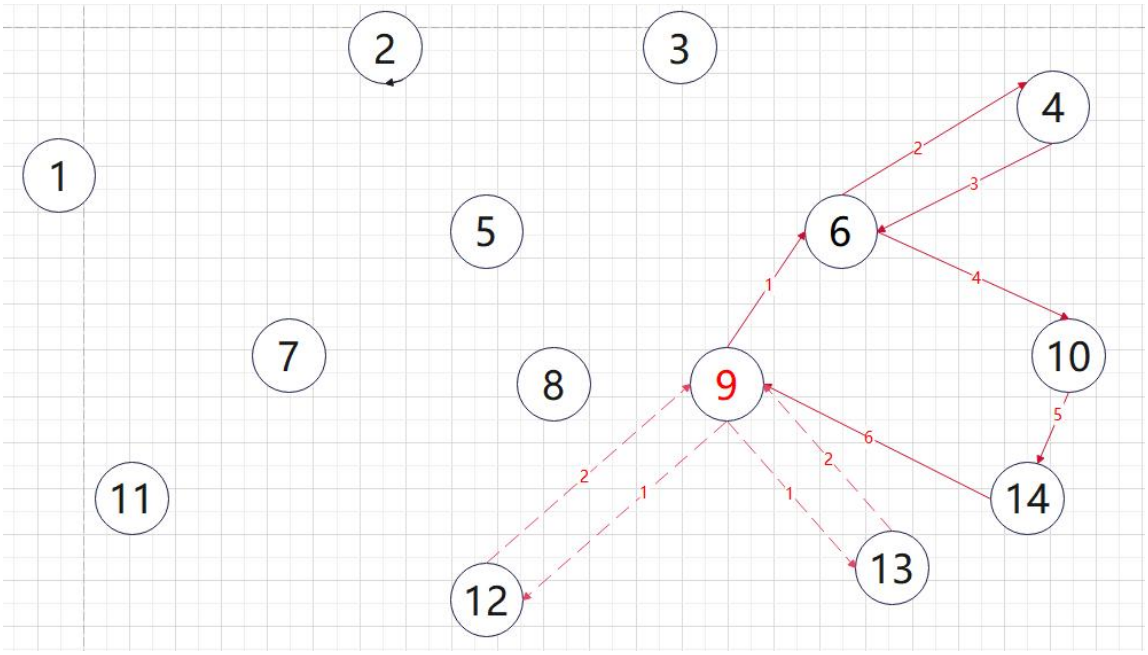

图 8 第一类地点顺序图

第二趟车辆配送最优方案总里程为 265 公里，最短的配送时间是 5.3 小时。

车辆行驶路径：

$$9 \to 8 \to 7 \to 5 \to 2 \to 3 \to 5 \to 9$$

无人机行驶路径：

$$2 \to 1 \to 2 \quad 7 \to 11 \to 7$$

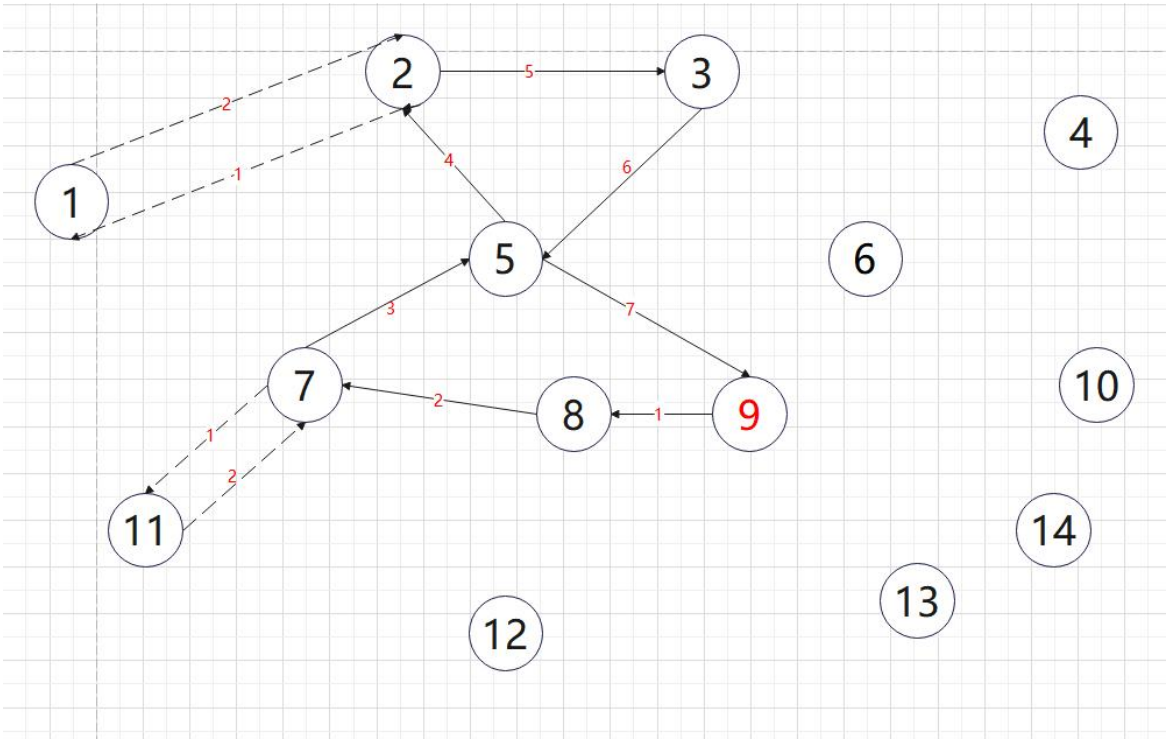

图 9 第二类地点顺序图

总的车辆最优方案总里程 451 公里，总的最短配送时间是 9.02

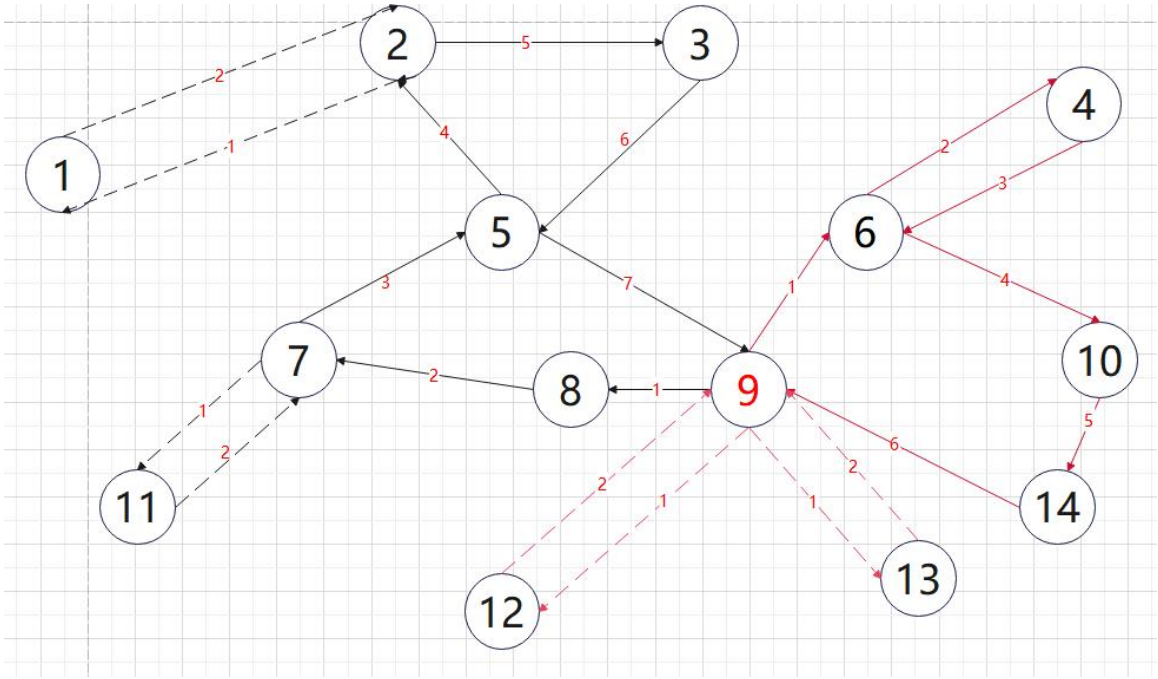

图 10 总地点顺序图

5.4 问题四的模型建立和求解

5.4.1 模型求解

通过聚类算法，我们可以将现有的所有地点分为两堆，这两堆的总距离所有情况下最小的。

至此，我们已经将多个点的问题回归到了几个点的问题，故我们可以对每堆做一个整体配送，依照题三的求解，得出该堆下的最小时间，并取两堆中时间最大的，得到一个总的时间。

通过多次迭代，我们可以将时间短的保留，最后总时间趋于平稳，得出最终结论。

通过以下 Matlab 程序，我们可以得出结论，两个最佳的应急配送点是 21 和 22，配送时间为 5.4 小时。

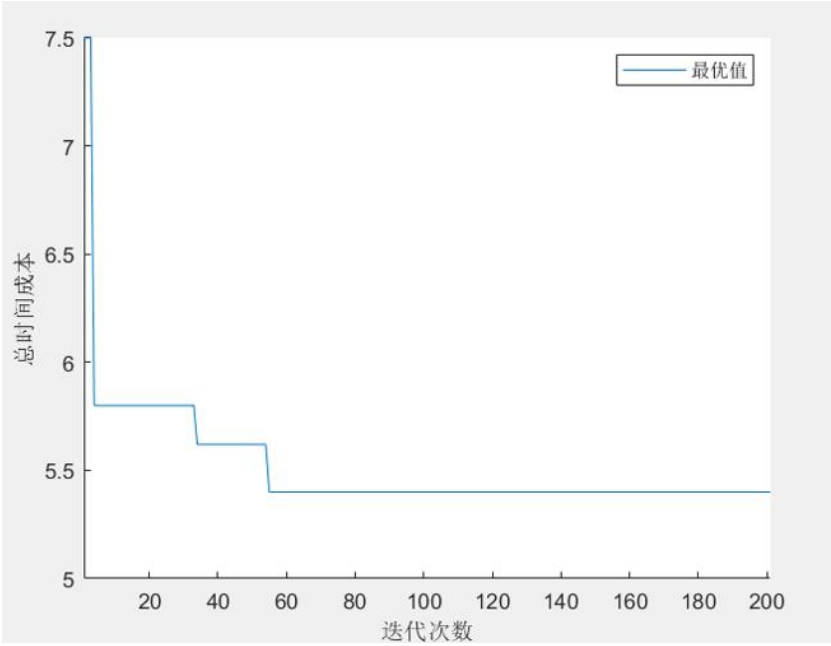

图 5 迭代次数与时间关系

我们通过计算，得出，以 21 为物资集中点，车运行的轨迹为:

$$21 \to 16 \to 12 \to 11 \to 15 \to 19 \to 24 \to 25 \to 20 \to 21 \to 17 \to 21$$

无人机的运行轨迹为：

$$10 \to 14 \to 13 \to 14$$

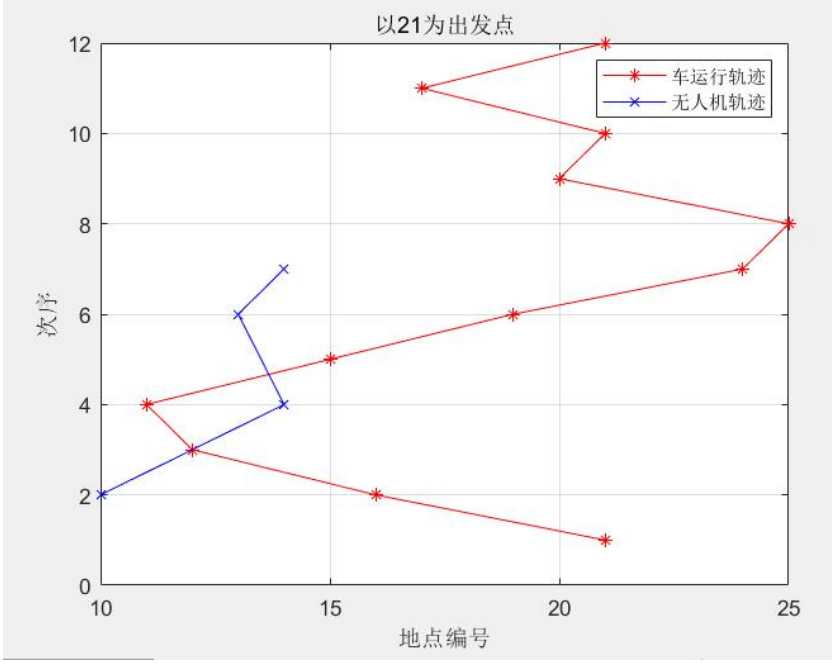

图 6 21 为出发点的轨迹图

以 22 为物资集中点，车运行的轨迹是

22 → 16 → 22 → 23 → 8 → 12 → 27 → 29 → 6 → 11 → 30 → 4 → 20 → 9 → 26 → 1 → 5 → 21 → 10 → 24 → 13 → 14 → 28 → 25 → 2 → 19 → 3 → 17 → 7

无人机运行的轨迹为：

$$21 \to 16 \to 21 \to 22 \to 21 \to 20 \to 20 \to 25 \to 24 \to 25$$

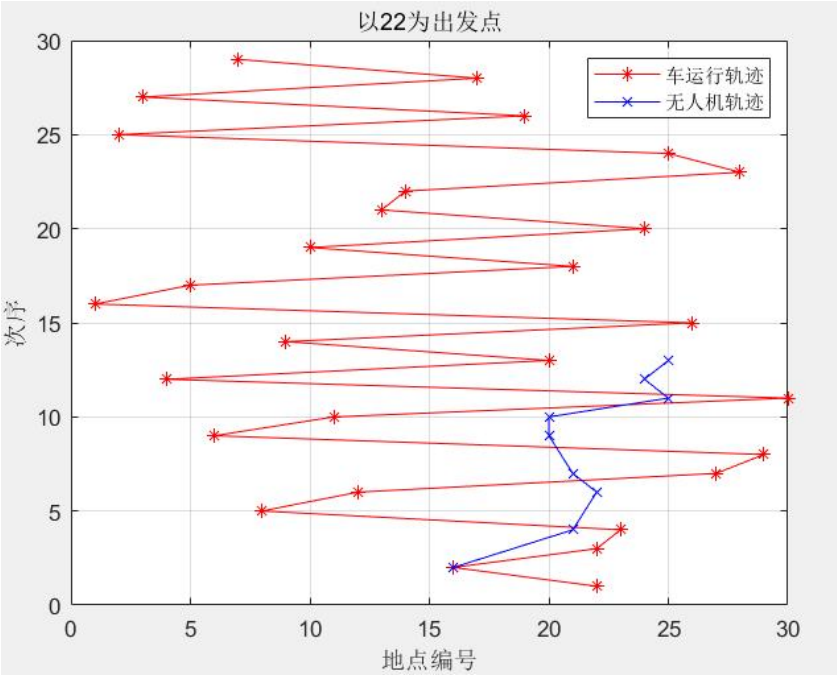

图 7　22 为出发点的轨迹图

六 模型的评价及改进

6.1　模型一的推广与改进

该算法可以运用于结点数目较少生活实际问题中。该算法始终是将目前最短的路径拿进来，来构成当前是最优的，从而来构建整体的最短路径，该算法的实质是贪心，不断挑选当前是最好的。

然而，对于结点数目较多或者边数较多的题目，我们运用该算法就无法得到最优解，因为，两点间的最短距可能距离第三点较远，所需要的代价更高。因此，我们可以运用动态规划去改进该算法，不仅仅考虑当前位置的最小点，还需要考虑，下一路径是全局的最小路径。

6.2　模型二的推广与改进

考虑到在第一题的基础上加入了无人机的配送，无人机的配送会影响车辆配送的路径，同时无人机的最大航城距离，最大载重限制无人机的配送路径，故我们要计算哪些地点可以由无人机配送。以无人机配送的地点为染色体，使用遗传算法求解某一组合下目标函数，即完成一次整体配送所需要的时间最短。如果无人机配送的地点在最佳配送

路线中出现，则此地点无需无人机配送，修改种群中的个体，一些地点无需无人机配送，修改该个体的染色体通过遗传算法多次迭代计算，再计算车辆额外等待的时间=无人机在两地点之间飞行时间-车辆最佳配送路线上的相同两地点之间行驶所花费的时间，取适应值最高即为所需结果。

6.3 模型三的推广与改进

该模型在第二问的基础上增加了限制条件，车辆最大载重从 1000kg 变成 500kg，这已经不允许车辆不回到物资聚集点，则我们需要对地方进行分组，使用聚类算法将所有地点分为两类，使得两类组内距离最小，利用问题二的代码对两类组进行处理，那么运行两次后的结果即为所需结果。

6.4 模型四的推广与改进

该模型在第三问的基础上增加限制条件，物资集中地点可变，图 3 中有 30 个地点，计划设置两个应急物资集中地点，配送车辆的最大载重量为 500 千克，我们同第三题一样使用两次聚类算法将所有地点分为 2 类，使得每类组内距离最小，先随机找一个解，作为初始可行解。故初始解会影响聚类的速度和最终聚类结果（陷入局部最优解）。可以通过多次随机选取初始解找到较优的结果类内寻找新类心，对于一片数据点来说，数据点的中心（坐标的平均值），离这些点的总距离最短。若类心改变，则总距离变化，以总距离的变化来判断类心是否有变化。最后用遗传算法的基础代码进行多次迭代，计算适应度得出适应度最高的即为所需结果。

七 参考文献

[1] 卢天毅，基于改进人工蜂群与 Floyd 算法的多机器人运输路径规，科技与创新，2021 年第 15 期：39-45，2021

[2] JeffCoding，弗洛伊德（Floyd）算法求图的最短路径，https://blog.csdn.net/jeffleo/article/details/53349825， 2022 年 5 月 29 日

[3] 胡大伟，陈海妹，车辆与无人机混合编队的路径优化问题模型构建，长安大学学报（自然科学版），2020 年 1 月，第 4 1 卷第 1 期：78-89

[4] 李凌军，基于聚类算法的数据情报挖掘与分析，信息技术，2020 年第 5 期：123-127

附录

附录一：

代码介绍：问题一的 Matlab 代码。

```
X=xlsread('附件1.xlsx')
X(1,:)=[]
X(:,1)=[]
X(isnan(X)) = 0
a=length(X) %距离矩阵维度,总共的点数
Y=zeros(a)    %构建可达矩阵
for i = 1:a
    for j = 1:a
        if X(i,j)==0
            Y(i,j)=0
        else
            Y(i,j)=1
        end
    end
end
next=9          %9号点开始
sum_s=0         %运行的总路程
point=[next]       %记录点的集合
for i = 1:a-1
    next_old = next
    next=seek(next,X,Y)
    point=[point,next]
    sum_s=sum_s+X(next,next_old)
    [X,Y]=change(X,Y,next_old)    %返回改变了的可达矩阵和距离矩阵
end
Point

function next=seek(a,X,Y) %寻找下一距离最短的点，返回点的序号
n_d=length(X)
m=100 %最大值
d=0 %该最小值的点
for i= 1:n_d
    if Y(a,i)==1
        if m>X(a,i) && a~=i
            m=X(a,i)
            d=i
        end
```

```
    end
end
next=d

function [X,Y]=change(X,Y,next_old)
n=length(X)
for i = 1:n
    for j = i:n
        if Y(i,next_old)==1 && Y(j,next_old)==1 && Y(i,j)==0 && Y(j,i)==0
            Y(i,j)=1
            Y(j,i)=1
            X(i,j)=X(i,next_old)+X(j,next_old)
            X(j,i)=X(i,j)
        end
    end
end
X(next_old,:)=0
X(:,next_old)=0
Y(next_old,:)=0
Y(:,next_old)=0
```

附录二：

代码介绍：问题二的 Matlab 代码。

```
MaterialNeed=[12 90 24 15 70 18 150 50 30 168 36 44 42 13]; % 前三问的数据
PositionNum=length(MaterialNeed);
DistributionCenter=9;
DistributionCenterNum=length(DistributionCenter);
global Distance_1
global Distance_2_3
Distance_1=[];  % 问题1中点与点的距离（不包括无人机路径）
Distance_2_3=[];  % 问题2、3中增加的无人机的距离，也即无人机可以飞行队路径
VehicleCapacity=1000;
VehicleNum=DistributionCenterNum;
% 配送车辆行驶速度km/h
global VehicleSpeed
VehicleSpeed=50;
UAVCapacity=50;
UAVNum=VehicleNum;
global UAVSpeed
UAVSpeed=75;
UAVEnduranceTime=70/60;
UAVEnduranceDistance=UAVSpeed*UAVEnduranceTime;
```

```
% 加载已经处理好的数据
load('data_of_2022527B.mat')
% 配送车辆可行驶路径（配送车辆）
global distance_min_1
global path_min_1
global time_min_1
distance_min_1=[];
path_min_1={};
time_min_1=[];
%myfolyd算法（两点之间的路径最短问题）
for i=1:PositionNum
    for j=1:PositionNum
        [dist,path]=myfloyd(Distance_1,i,j);
        distance_min_1(i,j)=dist;
        time_min_1(i,j)=dist/VehicleSpeed;
        if j>=i
        path_min_1{i,j}=path;
        path_min_1{j,i}=flip(path);
        end
    end
end

distance_min_1;
path_min_1;
time_min_1;
%无人机
global distance_min_2
global path_min_2
global time_min_2
distance_min_2=[];
path_min_2={};
for i=1:PositionNum
    for j=1:PositionNum
        [dist,path]=myfloyd(Distance_2_3,i,j);
        distance_min_2(i,j)=dist; %距离
        time_min_2(i,j)=dist/UAVSpeed; %时间
        if j>=i
            path_min_2{i,j}=path;
            path_min_2{j,i}=flip(path);
        end
    end
end
distance_min_2;
path_min_2;
```

```
time_min_2;
record_path_min_2={};
record_distance_min_2=[];
for i=1:PositionNum
        record_path_min_2=[record_path_min_2;path_min_2(:,i)];
        record_distance_min_2=[record_distance_min_2;distance_min_2(:,i)];
end

a=find(record_distance_min_2>UAVEnduranceDistance);
record_path_min_2(a,:)=[];
record_distance_min_2(a,:)=[];

global path_end
global distance_end
path_temp=record_path_min_2;
path_end={};
distance_temp=record_distance_min_2;
distance_end=[];
while ~isempty(path_temp)
    pos_last=path_temp{1}(end);
    b=find(Distance_2_3(pos_last,:)~=inf);
    distance_sum=[];
    for i=1:length(b)
        distance_new=distance_temp(1)+Distance_2_3(pos_last,b(i));
        distance_sum=[distance_sum;distance_new];
        if distance_new<=UAVEnduranceDistance
            distance_temp=[distance_temp;distance_new];
            path_new=[path_temp{1},b(i)];
            path_temp=[path_temp;path_new];
            path_end=[path_end;path_new];
            distance_end=[distance_end;distance_new];
        end
    end

    path_temp(1)=[];
    distance_temp(1)=[];
end
path_end_array=[];
for i=1:length(path_end)
    path_end_array(i,1:length(path_end{i}))=path_end{i};
end
path_end=path_end_array;

global UAVDeliveryPositionMaybe
```

```
UAVDeliveryPositionMaybe=find(MaterialNeed<=UAVCapacity);
UAVDeliveryPositionMaybe(UAVDeliveryPositionMaybe==9)=[];
global UAVNoFlyPosition
UAVNoFlyPosition=find(MaterialNeed>UAVCapacity);
UAVNoFlyPosition=sort(unique([UAVNoFlyPosition,9]));

popsize=200; %群体大小
chromlength=8; %字符串长度（个体长度，因为无人机可以配送的地点有8个）
pc=0.001; %交叉概率
pm=0.0001; %变异概率
Tmean=10; %  适应度（配送时间）


chromlength=length(UAVDeliveryPositionMaybe);
pop=ycsf_initpop(popsize,chromlength);

for i=1:5 %迭代次数
    [objvalue,~,~,~]=ycsf_calobjvalue(pop);
    Tmean=mean(objvalue);
    fitvalue=ycsf_calfitvalue(objvalue,Tmean);
    [newpop]=ycsf_selection(pop,fitvalue);
    pop=newpop;
end
[bestindividual,bestfit]=ycsf_best(pop,fitvalue);
[objvalue,dist_vehicle,path_vehicle,path_UAV]=ycsf_calobjvalue(bestindividual);
while isempty(path_UAV)

[objvalue,dist_vehicle,path_vehicle,path_UAV]=ycsf_calobjvalue(bestindividual);
end

disp('车辆最优配送方案path_vehicle=')
path_vehicle{1}
disp('无人机最优配送方案（第一个数字是由无人机配送的地点，之后的是无人机起飞、途径和降落的地点）path_UAV=')
path_UAV{1:end}
disp('车辆最优配送方案的总里程dist_vehicle=')
dist_vehicle
disp('最短配送时间MinTime=')
MinTime=objvalue

save ques2_result.mat path_vehicle path_UAV dist_vehicle MinTime

n=size(a,1); path=zeros(n);
for k=1:n
```

```
    for i=1:n
        for j=1:n
            if i==j
                a(i,j)=inf;
                path(i,j)=i;
            elseif a(i,j)>a(i,k)+a(k,j)
                a(i,j)=a(i,k)+a(k,j);
                path(i,j)=k;
            end
        end
    end
end
dist=a(sb,db);
parent=path(sb,:);
parent(parent==0)=sb;
mypath=db; t=db;
while t~=sb
        p=parent(t); mypath=[p,mypath];
        t=p;
end

%初始化
function pop=ycsf_initpop(popsize,chromlength)
pop=[];
for t=1:popsize
    ChromosomeCode=round(rand(1,chromlength));
    pop=[pop;ChromosomeCode];

end

end

function [objvalue,dist_vehicle,path_vehicle,path_UAV]=ycsf_calobjvalue(pop)
global distance_min_1
global path_min_1
global time_min_1
global distance_min_2
global path_min_2
global time_min_2
global path_end
global distance_end
global Distance_1
global UAVDeliveryPositionMaybe
global UAVNoFlyPosition
```

```
global VehicleSpeed
global UAVSpeed

objvalue=zeros(size(pop,1),1); % 即最短配送时间
dist_vehicle=zeros(size(pop,1),1); % 车辆最优配送方案的总里程
path_vehicle=cell(size(pop,1),1); % 即车辆最优配送方案
path_UAV=cell(size(pop,1),1); % 即无人机最优配送方案

for i = 1:size(pop,1)   % 计算最短时间
    a=UAVDeliveryPositionMaybe(pop(i,:)==0);
    b=UAVNoFlyPosition;
    VehicleDelivery=sort([a,b]);
    UAVDelivery=UAVDeliveryPositionMaybe(pop(i,:)==1);
    n = length(VehicleDelivery);
    xy = rand(n,2);
    salesmen = 1;   % 旅行者，即配送车辆数
    min_tour = n; % 一个车辆最少可以配送几个地点
    pop_size = 20; % 种群数
    num_iter = 5e1;   % 迭代数

    dmat=distance_min_1(VehicleDelivery,VehicleDelivery);
    [opt_rte,~,~] = mtspf_ga(xy,dmat,salesmen,min_tour,pop_size,num_iter,0,0);
    opt_rte_restore=VehicleDelivery(opt_rte);
    opt_rte=opt_rte_restore;

    path=[VehicleDelivery(1),opt_rte];
    a=find(path==9,1);
    path=[path(a:end),path(1:a-1),9];
    for j=1:length(path)-1
        if Distance_1(path(j),path(j+1))==inf
            path=[path(1:j-1),path_min_1{path(j),path(j+1)},path(j+2:end)];
        end
    end
    path_vehicle{i}=path;
    distance_total=0;
    for j=1:length(path)-1
        distance_total=distance_total+distance_min_1(path(j),path(j+1));
    end
    dist_vehicle(i)=distance_total;
    time_total=distance_total/VehicleSpeed;
    record=[];
    for j=1:length(UAVDelivery)
        if ~isempty(find(path==UAVDelivery(j), 1))
            record=[record,j];
```

```
        end
    end
    UAVDelivery(record)=[];

    pop_change=pop(i,:);
    for j=1:length(UAVDeliveryPositionMaybe)
        if ~isempty(find(UAVDeliveryPositionMaybe(j)==UAVDelivery, 1))
            pop_change(j)=1;
        else
            pop_change(j)=0;
        end
    end
    pop(i,:)=pop_change;

    % 初始化包含本次无人机配送的地点的无人机路径
    path_mid=[UAVDelivery',zeros(length(UAVDelivery),size(path_end,2))];
    path_UAV_mid=[UAVDelivery',ones(length(UAVDelivery),2).*9];
    dist_UAV_mid=[UAVDelivery',zeros(length(UAVDelivery),1)];
    time_UAV_mid=[UAVDelivery',zeros(length(UAVDelivery),1)];
    for j=1:length(UAVDelivery)
        for p=1:length(path)-1
            for q=1:length(path_end)
                b=path_end(q,2:find(path_end(q,:)==0,1)-2);
                c=sum(b==UAVDelivery(j));
                start_pos=path_end(q,1);
                end_pos=path_end(q,find(path_end(q,:)==0,1)-1);
                if   c>0 && ~isempty(find(path==start_pos,1)) &&
~isempty(find(path==end_pos,1)) &&
find(path==start_pos,1)<=find(path==end_pos,1)
                    path_UAV_mid(j,2)=start_pos;
                    path_UAV_mid(j,3)=end_pos;
                    path_mid(j,2:end)=path_end(q,:);
                    dist_UAV_mid(j,2)=distance_end(q,:);
                    time_UAV_mid(j,2)=distance_end(q,:)/UAVSpeed;

                    flag=1;
                    break
                end
            end
            if flag==1
                break
            end
        end
        if flag==1
```

```
                continue
            end
        end
        start_end=[];
        for p=1:size(path_UAV_mid,1)
            for q=2:3
                start_end=[start_end,find(path==path_UAV_mid(p,q),1)];
            end
        end

        if sort(start_end)==start_end
            path_UAV{i}=path_mid;
            VehicleWaitTime=0;
            for j=1:length(UAVDelivery)
                time1=time_UAV_mid(j,2);
               path_mid_j=path_mid(j,2:find(path_mid(j,:)==0,1)-1);
               time2=0;
               for m=1:length(path_mid_j)-1
                    time2=time2+time_min_1(path_mid_j(m),path_mid_j(m+1));
               end
                time=time1-time2;
                if time>0
                     VehicleWaitTime=VehicleWaitTime+time;
                end
            end
            VehicleWaitTime;
            Time_Total=time_total+VehicleWaitTime;
            objvalue(i)=Time_Total;
        else
            path_UAV{i}=[];
            objvalue(i)=20; %sum(sum(time_min_1(time_min_1~=inf)));
        end
end
end

%选择复制
function [newpop]=ycsf_selection(pop,fitvalue)
totalfit=sum(fitvalue);
fitvalue=fitvalue/totalfit;
fitvalue=cumsum(fitvalue);
[px,py]=size(pop);
ms=sort(rand(px,1));
fitin=1;
newin=1;
```

```
while newin<=px
    if(ms(newin))<fitvalue(fitin)
        newpop(newin,:)=pop(fitin,:);
        newin=newin+1;
    else
        fitin=fitin+1;
    end
end

function [bestindividual,bestfit]=ycsf_best(pop,fitvalue)
[px,py]=size(pop);
bestindividual=pop(1,:);
bestfit=fitvalue(1);
for i=2:px
    if fitvalue(i)>bestfit
        bestindividual=pop(i,:);
        bestfit=fitvalue(i);
    end
end

function fitvalue=ycsf_calfitvalue(objvalue,Tmean)
[px,py]=size(objvalue);
for i=1:px
    if Tmean-objvalue(i)>0
        temp=Tmean-objvalue(i);
    else
        temp=0.0;
    end
    fitvalue(i)=temp;
end
fitvalue=fitvalue';
```

附录三:

代码介绍：问题三的 Matlab 代码。

```
function [center,fenlei]=K_mean(k,Distance_1,distance_min_1)
k ;
point_num=size(Distance_1,1);
point_num; % d为样本点的个数
center = zeros(k,1);
suiji = randperm(point_num);
center = [6,7];

ddcs = 50; %设置迭代次数，若迭代超过一定次数则停止
zjl = zeros(1,ddcs); %总距离，所有类心到类内各点距离之和。
```

```
for q = 1:1:ddcs
    distences = zeros(k,point_num);
    dist=distance_min_1;
    dist(dist==inf)=0;
    for i = 1:1:k
   for j = 1:1:point_num
        distences(i,j) = dist(center(i),j);
   end
    end
    guilei = zeros(1,point_num);
    for i = 1:1:point_num
        guilei(1,i) = min(find(distences(:,i) == min(distences(:,i))));
    end
    DistSum=[];
    for i = 1:k
        a = find(guilei(1,:) == i);
        for j=1:length(a)
            distsum=0;
            for m=1:length(a)
                distsum=distsum+dist(a(j),a(m));
            end
            DistSum(i,j)=distsum;
        end
        DistSum(DistSum==0)=inf;
        center(i)=a(find(DistSum(i,:)==min(DistSum(i,:)),1));
        fenlei{i}=a;
    end
    DistSum(DistSum==inf)=0;
    zjl(1,q) = sum(sum(DistSum));
    if q > 1
        if zjl(1,q) == zjl(1,q - 1)
            break
        end
    end
end

%分组
k=2;
[center,fenlei]=K_mean(k,Distance_1,distance_min_1);

% 第一趟
if isempty(find(fenlei{1}==9,1))
    fenlei{1}=[fenlei{1},9];
```

```
end
[path_vehicle_1,path_UAV_1,dist_vehicle_1,MinTime_1]=ques2method(fenlei{1});
disp('第一趟车辆最优配送方案path_vehicle=')
path_vehicle_1{1}
disp('第一趟无人机最优配送方案（第一个数字是由无人机配送的地点，之后的是无人机起飞、途径和降落的地点）path_UAV=')
path_UAV_1{1:end}
disp('第一趟车辆最优配送方案的总里程dist_vehicle=')
dist_vehicle_1
disp('第一趟最短配送时间MinTime=')
MinTime_1
% 第二趟
if isempty(find(fenlei{2}==9,1))
    fenlei{2}=[fenlei{2},9];
end
[path_vehicle_2,path_UAV_2,dist_vehicle_2,MinTime_2]=ques2method(fenlei{2});
disp('第二趟车辆最优配送方案path_vehicle=')
path_vehicle_2{1}
disp('第二趟无人机最优配送方案（第一个数字是由无人机配送的地点，之后的是无人机起飞、途径和降落的地点）path_UAV=')
path_UAV_2{1:end}
disp('第二趟车辆最优配送方案的总里程dist_vehicle=')
dist_vehicle_2
disp('第二趟最短配送时间MinTime=')
MinTime_2

save ques3_result.mat path_vehicle_1 path_UAV_1 dist_vehicle_1 MinTime_1
path_vehicle_2 path_UAV_2 dist_vehicle_2 MinTime_2
```

附录四:

代码介绍：问题四的 Matlab 代码。

```
k=2;
[center,fenlei]=K_mean(k,Distance_4_1,distance_min_1);



k=2;
[center,fenlei]=K_mean(k,Distance_4_1,distance_min_1);
Distance=Distance_4_1(fenlei{1},fenlei{1});
distance_min=distance_min_1(fenlei{1},fenlei{1});
[center_2,fenlei_2]=K_mean(k,Distance,distance_min);
center_2(1)=fenlei{1}(center_2(1));
center_2(2)=fenlei{1}(center_2(2));
```

```
fenlei_2{1}=fenlei{1}(fenlei_2{1});
fenlei_2{2}=fenlei{1}(fenlei_2{2});


k=2;
Distance=Distance_4_1(fenlei{2},fenlei{2});
distance_min=distance_min_1(fenlei{2},fenlei{2});
[center_3,fenlei_3]=K_mean(k,Distance,distance_min);
center_3(1)=fenlei{2}(center_3(1));
center_3(2)=fenlei{2}(center_3(2));

fenlei_3{1}=fenlei{2}(fenlei_3{1});
fenlei_3{2}=fenlei{2}(fenlei_3{2});

% 第一个物资配送范围
% 第一趟
[path_vehicle_1,path_UAV_1,dist_vehicle_1,MinTime_1]=ques2method(fenlei_2{1
},center_2(1));

disp('第一个物资配送范围第一趟车辆最优配送方案path_vehicle=')
path_vehicle_1{1}
disp('第一个物资配送范围第一趟无人机最优配送方案path_UAV=')
path_UAV_1{1:end}
disp('第一个物资配送范围第一趟车辆最优配送方案的总里程dist_vehicle=')
dist_vehicle_1
disp('第一个物资配送范围第一趟最短配送时间MinTime=')
MinTime_1


% 第二趟
[path_vehicle_2,path_UAV_2,dist_vehicle_2,MinTime_2]=ques2method(fenlei_2{2
},center_2(2));

disp('第一个物资配送范围第二趟车辆最优配送方案path_vehicle=')
path_vehicle_2{1}
disp('第一个物资配送范围第二趟无人机最优配送方案path_UAV=')
path_UAV_2{1:end}
disp('第一个物资配送范围第二趟车辆最优配送方案的总里程dist_vehicle=')
dist_vehicle_2
disp('第一个物资配送范围第二趟最短配送时间MinTime=')
MinTime_2

% 第二个物资配送范围
% 第一趟
```

```
[path_vehicle_3,path_UAV_3,dist_vehicle_3,MinTime_3]=ques2method(fenlei_3{1
},center_3(1));

disp('第二个物资配送范围第一趟车辆最优配送方案path_vehicle=')
path_vehicle_3{1}
disp('第二个物资配送范围第一趟无人机最优配送方案path_UAV=')
path_UAV_3{1:end}
disp('第二个物资配送范围第一趟车辆最优配送方案的总里程dist_vehicle=')
dist_vehicle_3
disp('第二个物资配送范围第一趟最短配送时间MinTime=')
MinTime_3


% 第二趟
[path_vehicle_4,path_UAV_4,dist_vehicle_4,MinTime_4]=ques2method(fenlei_3{2
},center_3(2));

disp('第二个物资配送范围第二趟车辆最优配送方案path_vehicle=')
path_vehicle_4{1}
disp('第二个物资配送范围第二趟无人机最优配送方案path_UAV=')
path_UAV_4{1:end}
disp('第二个物资配送范围第二趟车辆最优配送方案的总里程dist_vehicle=')
dist_vehicle_4
disp('第二个物资配送范围第二趟最短配送时间MinTime=')
MinTime_4

save ques4_result.mat path_vehicle_1 path_UAV_1 dist_vehicle_1 MinTime_1
path_vehicle_2 path_UAV_2 dist_vehicle_2 MinTime_2...
        path_vehicle_3 path_UAV_3 dist_vehicle_3 MinTime_3 path_vehicle_4
path_UAV_4 dist_vehicle_4 MinTime_4
```